

DECLARATION

I/We declare that this project report titled ”**BODHYAAI: AN AI-POWERED MENTORING AND ACADEMIC RISK DETECTION PLATFORM**” submitted in partial fulfillment of the degree of **Bachelor of Engineering in Computer Science and Engineering (Artificial Intelligence and Machine Learning)** is a record of original work carried out by me/us under the supervision of **Ms. Revathi S**, and has not formed the basis for the award of any other degree, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Signature of the student(s)

Name

Signature

Poorna Teja Reddy K 1CR23CI403

Manikanta 1CR23CI402

C Karthik 1CR22CI013

Sree Ranganath 1CR22CI053

ACKNOWLEDGMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible. Success is the epitome of hard work and perseverance; however, steadfast encouragement and guidance play a crucial role in achieving it. Hence, it is with deep gratitude that we acknowledge all those whose support and guidance served as a beacon of light and crowned our efforts with success.

We would like to express our sincere thanks to Dr. Sanjay Jain, Principal, CMR Institute of Technology, Bengaluru, for providing an excellent academic environment and for his constant encouragement throughout the Bachelor of Engineering program.

We extend our heartfelt gratitude to Dr. Shyam P. Joy, Head of the Department of Artificial Intelligence and Machine Learning, CMR Institute of Technology, Bengaluru, for his valuable guidance, motivation, and constructive suggestions that greatly contributed to the successful completion of this project.

We consider it a privilege and an honour to express our sincere gratitude to our project guide, Ms. Revathi S, Assistant Professor, Department of Artificial Intelligence and Machine Learning, CMR Institute of Technology, Bengaluru, for her continuous support, expert guidance, and encouragement throughout the duration of this project work.

We also extend our thanks to all the faculty members and non-teaching staff of the Department of Artificial Intelligence and Machine Learning for their cooperation and assistance, which helped us immensely during the course of this project.

Finally, we would like to thank our family members for their constant encouragement, support, and motivation. We also acknowledge everyone who has directly or indirectly contributed to the successful completion of this project.

**POORNA TEJA REDDY K
MANIKANTA
C KARTHIK
SREE RANGANATH**

ABSTRACT

BodhyaAI is an AI-powered academic mentoring and risk detection platform designed to identify academically vulnerable students at an early stage. The system integrates machine learning-based academic and dropout risk prediction with cognitive profiling using the Big Five Inventory (BFI-44) psychological model. To ensure transparency and trust in decision-making, BodhyaAI incorporates Explainable Artificial Intelligence (XAI) techniques such as SHAP and LIME, enabling mentors to understand the key factors influencing each prediction.

In addition to predictive analytics, the platform includes a generative MentorBot built using a Retrieval-Augmented Generation (RAG) framework, which provides personalized academic guidance and actionable mentoring recommendations tailored to individual student profiles. The overall system follows a scalable microservices architecture with dedicated services for risk prediction, explainability, cognitive insights, and LLM-driven mentoring.

Experimental evaluation demonstrates strong prediction accuracy, interpretable insights, and meaningful student-specific guidance, validating the effectiveness of the proposed approach. BodhyaAI bridges the gap between data-driven analytics and human-centered mentoring, offering an intelligent, transparent, and scalable solution for modern educational institutions.

Contents

Acknowledgment	ii
Abstract	iii
Table of Contents	iv
List of Figures	viii
List of Tables	ix
Abbreviations	x
1 INTRODUCTION	1
1.1 Background and Context	1
1.2 Challenges in Academic Risk Prediction and Mentoring	2
1.2.1 Limited Scalability	2
1.2.2 Delayed Identification of Academic Risk	2
1.2.3 Opaque Machine Learning Models	2
1.2.4 Insufficient Inclusion of Cognitive and Psychological Attributes	3
1.2.5 Lack of Actionable and Personalized Mentoring	3
1.2.6 Fragmented Technological Ecosystem	3
1.3 Importance of Early Intervention Systems in Education	3
1.4 Motivation for Developing BodhyaAI	3
1.4.1 Transparent AI in Education	4
1.4.2 Integration of Cognitive Profiling	4
1.4.3 From Prediction to Action	4
1.4.4 Scalable and Modular Architecture	4
1.4.5 Reduction of Mentor Workload	4
1.5 Problem Statement	4
1.6 Project Objectives	4
1.7 Scope of the Project	5
1.7.1 Included in Scope	5
1.7.2 Excluded from Scope	5
1.8 Methodology Overview	5
2 LITERATURE REVIEW	6
2.1 Introduction	6
2.2 Evolution of Educational Data Mining and Learning Analytics	7

2.3	Machine Learning in Academic Performance Prediction	8
2.3.1	Predictive Frameworks and Methodologies	8
2.3.2	Limitations of Existing ML Approaches	9
2.4	Cognitive and Psychological Models in Education	9
2.4.1	Big Five Personality Traits (BFI-44)	9
2.4.2	Cognitive Insights for Academic Mentoring	9
2.5	Explainable Artificial Intelligence (XAI) in Education	9
2.5.1	Importance of XAI	9
2.6	Generative AI for Personalized Academic Mentoring	9
2.7	Comparative Review of Existing Academic Risk Systems	10
2.8	Research Gaps Identified	10
3	METHODOLOGY	11
3.1	Overview	11
3.2	Data Collection	12
3.3	Dataset Description	13
3.4	Data Preprocessing	13
3.4.1	Missing Value Imputation	14
3.4.2	Outlier Detection and Removal	14
3.4.3	Normalization and Scaling	14
3.5	Feature Engineering	14
3.5.1	Temporal Trend Analysis	14
3.5.2	Risk Index Calculation	15
3.6	Machine Learning Model Architecture	15
3.6.1	Mathematical Formulation of XGBoost	15
3.6.2	Hyperparameter Tuning	16
3.7	Explainable AI (XAI) Methodology	17
3.7.1	SHAP (SHapley Additive exPlanations)	17
3.7.2	LIME (Local Interpretable Model-agnostic Explanations) . . .	18
3.8	Cognitive Profiling Methodology	18
3.8.1	Trait Scoring	18
3.9	Generative MentorBot Methodology	18
3.9.1	Retrieval Mechanism	19
3.9.2	Generation	19
3.10	System Architecture and Integration	19
3.10.1	Microservices Decomposition	19
3.11	Summary	20
4	SYSTEM DESIGN AND IMPLEMENTATION	21
4.1	Introduction	21
4.2	System Architecture	21

4.3	Microservices Implementation Details	22
4.3.1	Risk Analysis Service (risk-svc)	22
4.3.2	Explainable AI Service (xai-svc)	23
4.3.3	Cognitive Profiling Service (cog-svc)	24
4.3.4	MentorBot Service (llm-svc)	24
4.4	Backend API Gateway Implementation	25
4.5	Database Design	25
4.5.1	Schema Definitions	25
4.6	User Interface Design	27
4.6.1	Login and Authentication	27
4.6.2	Student Dashboard	27
4.6.3	Mentor Dashboard	27
4.7	Testing and Validation	27
4.7.1	Unit Testing	28
4.7.2	Integration Testing	28
4.7.3	User Acceptance Testing (UAT)	28
4.8	Summary	28
5	IMPACT ASSESSMENT	29
5.1	Impact on Society	29
5.2	Impact on Environment	29
5.3	Impact on Technology Domain	30
6	RESULTS AND DISCUSSION	31
6.1	Introduction	31
6.2	Experimental Setup	31
6.2.1	Datasets	31
6.3	Model Performance Analysis	31
6.3.1	Hyperparameter Tuning Results	31
6.3.2	Academic Risk Prediction	32
6.3.3	Confusion Matrix Analysis	32
6.4	System Performance and Latency	33
6.5	Explainable AI Validation	34
6.5.1	SHAP vs. Feature Importance	34
6.6	Frontend Outcome and User Interface	34
6.6.1	Login Page	34
6.6.2	Student Dashboard	35
6.6.3	Mentor Dashboard	36
6.6.4	Explainable AI (SHAP) View	37
6.6.5	Cognitive Profile Visualization	38
6.6.6	Generative MentorBot Interface	39

6.7	Failure Analysis	40
6.8	Case Studies and Qualitative Analysis	40
6.8.1	Case Study A: The "High-Stress" Performer	40
6.8.2	Case Study B: The "Disengaged" Student	41
6.9	Summary	41
7	CONCLUSION AND FUTURE WORK	42
7.1	Conclusion	42
7.2	Practical Impact	43
7.3	Real-World Constraints and Deployment Challenges	43
7.4	Limitations	43
7.5	Future Work	44
7.5.1	Integration with Real Institutional Data	44
7.5.2	Adaptive and Continual Learning	44
7.5.3	Enhanced MentorBot Personalization	44
7.5.4	Mobile and Ubiquitous Access	44
7.5.5	Emotional and Behavioral Signal Integration	44
7.5.6	Collaborative Mentoring Tools	44
7.6	Final Remarks	45
	REFERENCES	45
	APPENDIX	46
A.1	User Manual	46
A.1.1	System Access	46
A.1.2	Student Module	46
A.1.3	Mentor Module	47
A.2	API Specifications	47
A.2.1	Risk Analysis Service	47
A.2.2	Explainable AI Service	48
A.2.3	Cognitive Service	48
A.3	Deployment Configuration	49
A.3.1	Prerequisites	49
A.3.2	Environment Variables	49
A.3.3	Docker Compose Setup	49
A.3.4	Deployment Steps	50
A.4	Source Code Listings	50
A.4.1	Risk Analysis Service (risk-svc/service.py)	50
A.4.2	Explainable AI Service (xai-svc/service.py)	52
A.4.3	Cognitive Service (cog-svc/service.py)	53

List of Figures

2.1	Conceptual Workflow of Educational Data Mining in AI-Based Mentorship Systems	7
3.1	Integrated workflow representing the Predict–Explain–Understand–Act methodology	12
3.2	BodhyaAI system architecture integrating prediction, explainability, cognitive profiling, and mentoring services	20
4.1	High-level system architecture of BodhyaAI	22
6.1	Normalized Confusion Matrix	33
6.2	BodhyaAI Login Interface	35
6.3	Student Dashboard with Risk Meter	36
6.4	Mentor Analytics Dashboard	37
6.5	Detailed Student Profile with SHAP Explanations	38
6.6	Cognitive Profile Radar Chart	39
6.7	MentorBot Chat Interface	40

List of Tables

2.1	Comparative Summary of Existing Methodologies in Academic Performance Prediction	8
2.2	Comparative Analysis of Existing Academic Risk Systems and BodhyaAI	10
3.1	Structure of the Academic Dataset	13
3.2	Structure of the Cognitive Dataset	13
3.3	Optimized XGBoost Hyperparameters	16
6.1	Hyperparameter Tuning Results (Top 5 Configurations)	32
6.2	Classification Report for Risk Prediction	32
6.3	Normalized Confusion Matrix (Rows = Actual)	33
6.4	System Latency Analysis (Average Response Time)	34

ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
BFI	Big Five Inventory
EDM	Educational Data Mining
JSON	JavaScript Object Notation
LIME	Local Interpretable Model-agnostic Explanations
LLM	Large Language Model
LMS	Learning Management System
ML	Machine Learning
RAG	Retrieval-Augmented Generation
SHAP	SHapley Additive exPlanations
UI	User Interface
XAI	Explainable Artificial Intelligence

Chapter 1

INTRODUCTION

The increasing complexity of higher education systems, coupled with rising student enrollments and diverse learning backgrounds, has created an urgent demand for scalable and data-driven academic mentoring frameworks. Traditional mentoring practices—largely dependent on manual observation, periodic assessments, and static performance indicators—are no longer sufficient for timely identification of academic difficulties nor for delivering personalized guidance at scale.

Modern educational institutions require intelligent systems capable of analyzing multidimensional student data, predicting academic risk at an early stage, offering transparent explanatory insights, and generating tailored interventions that support continuous academic progression. Advances in Artificial Intelligence (AI), particularly in Machine Learning (ML), Explainable Artificial Intelligence (XAI), and Generative AI (GenAI), provide new opportunities to address these requirements.

However, existing AI-enabled academic support systems continue to face significant limitations, including opaque model reasoning, lack of psychological context, insufficient personalization, and fragmented mentoring workflows. These constraints reduce mentor trust, hinder adoption, and limit the real-world impact of AI in educational environments.

To address these challenges, this project presents **BodhyaAI**, an explainable and generative AI-powered academic mentoring and risk detection platform designed to provide early risk identification, transparent reasoning, cognitive awareness, and actionable mentoring support within a unified and scalable architecture.

1.1 Background and Context

Student academic performance is influenced by a complex interplay of academic, behavioral, socioeconomic, psychological, and emotional factors. Conventional assessment systems primarily focus on measurable academic indicators such as grades, attendance, and examination scores, while often neglecting deeper cognitive dispositions, motivational drivers, and stress-related influences that significantly affect learning outcomes.

Research in Educational Data Mining (EDM) and Learning Analytics demonstrates that predictive models can effectively identify students at academic risk. Ensemble learning techniques such as Random Forest and XGBoost consistently achieve strong predictive accuracy. However, the absence of transparent explanation mechanisms limits their pedagogical usefulness, as mentors are unable to understand or trust the rationale behind predictions.

Simultaneously, studies in cognitive psychology highlight the importance of personality traits—such as conscientiousness, emotional stability, and openness—in shaping learning behavior and academic success. Integrating cognitive assessment tools like the Big Five Inventory (BFI-44) into academic analytics enables a more human-centered understanding of student performance.

Recent advances in Generative AI and Retrieval-Augmented Generation (RAG) further enable context-aware mentoring and personalized academic guidance, bridging the gap between risk detection and actionable intervention. BodhyaAI leverages these advancements to deliver an integrated, interpretable, and mentor-centric solution.

1.2 Challenges in Academic Risk Prediction and Mentoring

Despite the growing adoption of digital learning technologies, educational institutions continue to face multiple challenges in delivering effective and personalized mentoring.

1.2.1 Limited Scalability

Mentors are often responsible for large student cohorts, making continuous monitoring of individual academic progress difficult and prone to oversight.

1.2.2 Delayed Identification of Academic Risk

Traditional systems typically identify risk only after major assessments, resulting in reactive interventions rather than preventive support.

1.2.3 Opaque Machine Learning Models

Many ML-based systems function as “black boxes,” providing predictions without explaining contributory factors. This lack of transparency reduces mentor confidence and limits informed decision-making.

1.2.4 Insufficient Inclusion of Cognitive and Psychological Attributes

Despite strong empirical evidence, cognitive traits and psychological indicators are rarely incorporated into academic risk prediction frameworks.

1.2.5 Lack of Actionable and Personalized Mentoring

Existing platforms may detect risk but often fail to provide: Existing platforms may detect risk but often fail to provide personalized learning pathways, behaviorally aligned recommendations, and psychologically calibrated mentoring guidance.

1.2.6 Fragmented Technological Ecosystem

Tools for prediction, explainability, cognitive assessment, and mentoring frequently operate in isolation, reducing their collective effectiveness.

These challenges highlight the need for a unified, interpretable, and human-centered AI mentoring platform.

1.3 Importance of Early Intervention Systems in Education

Early identification of academic challenges has a direct impact on student retention, performance, and progression. Empirical studies show that predictive analytics combined with timely intervention significantly reduce dropout rates and improve learning outcomes.

An effective early intervention system must:

1. continuously monitor academic and behavioral indicators,
2. detect deviations from expected performance trends,
3. identify contributory risk factors,
4. incorporate cognitive and psychological attributes, and
5. deliver targeted and timely mentoring recommendations.

Such systems empower mentors to make informed decisions while enabling students to self-regulate their learning more effectively.

1.4 Motivation for Developing BodhyaAI

BodhyaAI was conceived to address critical gaps in existing academic analytics and mentoring systems.

1.4.1 Transparent AI in Education

Mentors require interpretable models that explain *why* a student is at risk, rather than merely presenting risk labels.

1.4.2 Integration of Cognitive Profiling

Personality-based insights improve mentoring alignment, engagement, and behavioral understanding.

1.4.3 From Prediction to Action

Generative AI enables the transition from risk identification to personalized, actionable mentoring recommendations.

1.4.4 Scalable and Modular Architecture

A microservices-based design supports independent deployment, maintainability, and institutional scalability.

1.4.5 Reduction of Mentor Workload

Automating analytical and interpretative tasks allows mentors to focus on meaningful academic interactions.

1.5 Problem Statement

Higher education institutions currently lack a unified platform capable of accurately predicting academic risk, explaining the reasoning behind predictions, integrating cognitive traits, and delivering personalized mentoring at scale.

Problem Statement:

Design and develop an explainable and generative AI-driven academic mentorship platform that integrates academic analytics, cognitive profiling, transparent model explanations, and personalized mentoring to support timely academic intervention in higher education.

1.6 Project Objectives

The primary objectives of this project are:

1. To develop machine learning models for academic and dropout risk prediction using XGBoost and Random Forest.

2. To implement Explainable AI techniques (SHAP and LIME) for transparent interpretation of predictions.
3. To perform cognitive profiling using the Big Five Inventory (BFI-44).
4. To design a generative mentoring system using RAG-enabled large language models.
5. To build an integrated mentorship dashboard combining academic, cognitive, and explanatory insights.
6. To deploy the platform using a scalable microservices architecture.

1.7 Scope of the Project

1.7.1 Included in Scope

The scope of the project includes academic and dropout risk prediction using ML models, cognitive profiling using BFI-44, and explainability using SHAP and LIME. It also encompasses a generative MentorBot using RAG-lite, a microservices-based backend architecture, and a web-based mentor dashboard using React.js.

1.7.2 Excluded from Scope

Excluded from the scope are mobile application development, LMS integration with institutional platforms, real-time biometric emotion recognition, and federated multi-campus learning systems.

1.8 Methodology Overview

BodhyaAI follows a structured four-phase mentoring cycle:

1. **Predict:** Academic risk classification using machine learning models.
2. **Explain:** Interpretation of predictions using XAI techniques.
3. **Understand:** Cognitive analysis using BFI-44 personality profiling.
4. **Act:** Personalized mentoring and guidance generation using Generative AI.

This end-to-end workflow distinguishes BodhyaAI from fragmented analytics solutions by offering a complete predictive-to-intervention loop.

Chapter 2

LITERATURE REVIEW

The integration of Artificial Intelligence (AI) into educational systems has grown substantially over the past decade, driven by the increasing need to enhance academic performance, personalize learning pathways, and enable early identification of students at academic risk. This chapter reviews the relevant literature across key domains that collectively form the foundation of the proposed BodhyaAI platform.

The literature review is organized into several key thematic areas, including Educational Data Mining and Learning Analytics, Machine Learning for Academic Risk Prediction, and Cognitive and Psychological Models in Education. Additionally, it covers Explainable Artificial Intelligence (XAI) Techniques and Generative AI for Personalized Academic Mentorship.

A critical synthesis of prior research reveals significant advancements as well as persistent limitations that motivate the design and development of BodhyaAI.

2.1 Introduction

Student academic performance is influenced by a combination of academic, behavioral, socio-economic, cognitive, and emotional factors. The increasing digitization of educational environments—through Learning Management Systems (LMS), online assessments, and institutional databases—has resulted in large volumes of educational data. This has given rise to the fields of Educational Data Mining (EDM) and Learning Analytics (LA), which aim to extract actionable insights from student data to support academic decision-making.

Despite their success in prediction, many existing systems operate as black-box models with limited transparency and negligible psychological context. Recent advances in Explainable AI (XAI), personality profiling, and Generative AI (GenAI) provide an opportunity to design more interpretable, human-centered, and actionable academic mentoring systems. The literature reviewed in this chapter establishes the theoretical and technical basis for BodhyaAI.

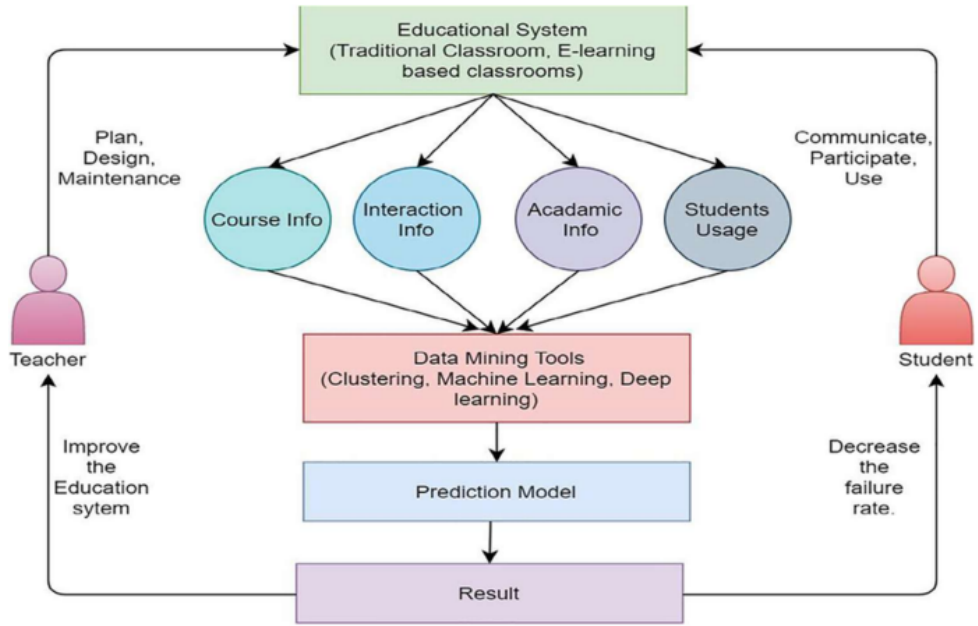


Figure 2.1: Conceptual Workflow of Educational Data Mining in AI-Based Mentorship Systems

Figure 2.1 illustrates how academic records, LMS interaction logs, assessment outcomes, and behavioral indicators are processed through AI and machine learning techniques to generate actionable insights that support early intervention and personalized mentoring.

2.2 Evolution of Educational Data Mining and Learning Analytics

Educational Data Mining focuses on discovering patterns in educational datasets to improve learning outcomes, institutional planning, and student support mechanisms. Early EDM research relied on statistical analysis and rule-based systems, whereas contemporary approaches increasingly employ ensemble learning, deep learning, and hybrid AI models.

Recent studies demonstrate the effectiveness of large-scale datasets comprising academic records, attendance logs, LMS interaction data, assessment histories, demographic attributes, and psychological indicators. For example: Recent studies utilize large-scale datasets comprising academic records and LMS interaction logs [?]. For instance, Balachandar and Venkatesh (2025) proposed the MSPP framework using deep learning and temporal modeling, while Johora et al. (2025) developed an explainable ML framework utilizing SMOTE [?] and ensemble methods. Existing Mentor Connect platforms emphasize communication but often lack predictive intelligence.

While EDM research highlights early risk detection, most systems lack interpretability and cognitive depth, limiting mentoring effectiveness.

2.3 Machine Learning in Academic Performance Prediction

Machine learning techniques are widely applied to predict academic outcomes such as grades, dropout risk, and subject failure probability. Traditional models include Logistic Regression, Decision Trees, SVM, and KNN. Recent research adopts ensemble models such as Random Forest, Gradient Boosting, and XGBoost.

2.3.1 Predictive Frameworks and Methodologies

Table 2.1: Comparative Summary of Existing Methodologies in Academic Performance Prediction

Authors	Methodology Used	Accuracy	Key Contribution
Johora et al. (2025)	SMOTE + RF + Gradient Boost + Stacking	86.38%	Explainable prediction using SHAP and LIME
Zulfiker et al.	Weighted Voting Classifier	81.73%	Course-grade prediction system
Anwar et al.	Artificial Neural Network (ANN)	88%	Learning capacity categorization
Janan et al.	SVM + Neuro-Fuzzy System	81.25%	Adaptive performance prediction
Fazil et al. (ASIST)	ConvBiLSTM with Attention	High F1-score	Sequential behavior analysis
Balachandar & Venkatesh (2025)	GNN + Deep Feature Engineering	76%	Temporal performance modeling

Table 2.1 shows that while modern ML models achieve high accuracy, interpretability and actionable mentoring remain limited.

2.3.2 Limitations of Existing ML Approaches

Key limitations include lack of interpretability, minimal cognitive inclusion, single-institution datasets, and weak linkage between prediction and intervention. BodhyaAI addresses these gaps through explainable, cognitive-aware, and generative mentoring.

2.4 Cognitive and Psychological Models in Education

Educational psychology research highlights the influence of personality traits on motivation, stress tolerance, learning behavior, and academic success.

2.4.1 Big Five Personality Traits (BFI-44)

The Big Five model [?] measures Openness, Conscientiousness, Extraversion, Agreeableness, and Neuroticism. Studies consistently show conscientiousness as a strong predictor of academic success [?], while high neuroticism correlates with stress and performance instability [?].

2.4.2 Cognitive Insights for Academic Mentoring

Cognitive profiling enables personalized mentoring styles, emotional awareness, and early identification of students requiring psychological support. BodhyaAI integrates BFI-44 profiling to enhance mentoring relevance.

2.5 Explainable Artificial Intelligence (XAI) in Education

XAI techniques such as SHAP [?] and LIME [?] provide transparency in academic risk prediction models [?].

2.5.1 Importance of XAI

XAI improves mentor trust, fairness [?], bias detection, identification of actionable factors [?], and effectiveness of feedback.

2.6 Generative AI for Personalized Academic Mentoring

LLMs such as GPT, Gemini, LLaMA, and Phi-3 [?] demonstrate strong conversational reasoning. RAG architectures [?, ?] reduce hallucination and enable grounded mentoring guidance [?]. These advances inspire BodhyaAI's MentorBot [?].

2.7 Comparative Review of Existing Academic Risk Systems

Table 2.2: Comparative Analysis of Existing Academic Risk Systems and BodhyaAI

System	Strengths	Limitations
LMS Analytics Systems	Tracks engagement and assessments	No prediction or psychological insights
MentorConnect Platform	Strong communication tools	No ML, XAI, or cognitive integration
Predictive ML Models	High accuracy	Black-box output, no mentoring
Deep Learning Models	Strong performance	Uninterpretable and computationally heavy
XAI-Integrated Models	Transparent predictions	No generative mentoring
BodhyaAI (Proposed)	Predictive, explainable, cognitive-aware, generative mentoring	Requires microservices infrastructure

Table 2.2 highlights BodhyaAI’s comprehensive integration of analytics, cognition, explainability, and mentoring.

2.8 Research Gaps Identified

The literature reveals the following gaps: The literature reveals significant gaps, primarily the lack of unified systems combining prediction, explanation, cognition, and mentoring. Most academic risk models suffer from insufficient interpretability and a limited focus on psychological determinants. Furthermore, there is a notable absence of generative mentoring frameworks, and many existing solutions exhibit poor scalability and modularity.

These gaps motivate the development of BodhyaAI as a holistic, explainable, and human-centered academic mentoring ecosystem.

Chapter 3

METHODOLOGY

The methodology adopted in developing **BodhyaAI** follows a comprehensive, multi-stage framework that integrates data preparation, machine learning-based academic risk prediction, cognitive profiling, explainable AI mechanisms, generative mentoring, and a scalable microservices-based deployment environment. The methodological foundation is informed by contemporary research in educational data mining, explainable artificial intelligence, and cognitive psychology. The proposed approach ensures that the system not only predicts academic risk with high accuracy but also provides transparent reasoning and psychologically informed mentoring recommendations.

3.1 Overview

The methodological design of BodhyaAI is centered around a structured four-phase mentoring cycle:

Predict → Explain → Understand → Act

This cycle ensures that raw student data is systematically transformed into actionable mentoring insights. The system first applies machine learning models to academic and behavioral data to identify students at risk of academic decline. These predictions are then interpreted using explainable AI techniques to provide feature-level justification. Cognitive profiling using the Big Five Inventory (BFI-44) introduces psychological context, and finally, a generative AI model produces personalized mentoring recommendations.

Figure 3.1 illustrates the integrated workflow of the Predict–Explain–Understand–Act methodology.

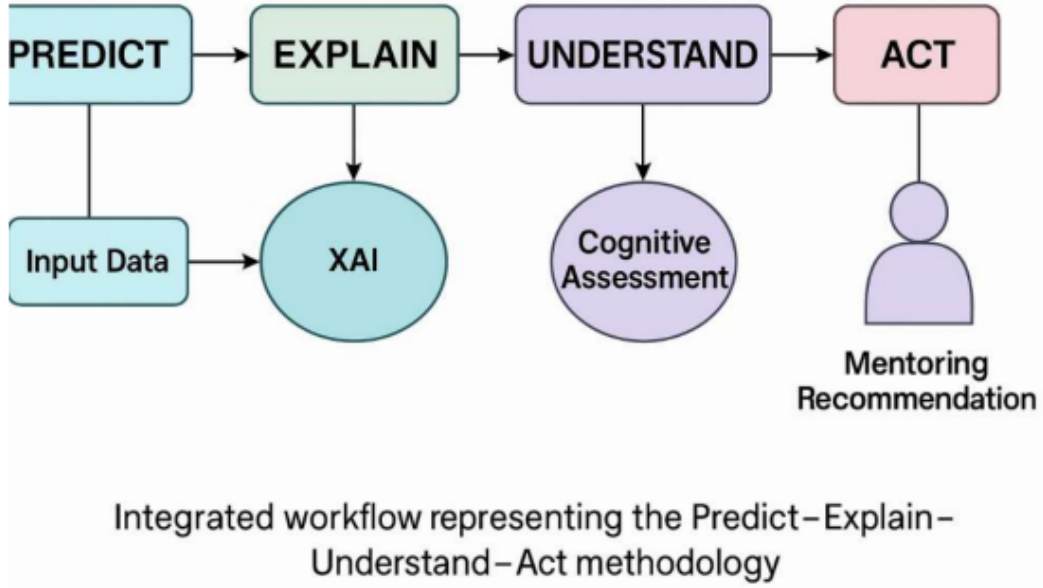


Figure 3.1: Integrated workflow representing the Predict-Explain-Understand-Act methodology

3.2 Data Collection

The development of BodhyaAI relies on two primary data streams: an academic performance dataset and a cognitive personality dataset. These datasets are synthesized to enable a multidimensional understanding of student performance.

The academic dataset consists of approximately 50,000 anonymized student records designed to represent realistic academic, behavioral, socioeconomic, and wellness-related factors. Features include semester-wise SGPA, cumulative CGPA, internal assessment scores, attendance percentages, study hours, backlog history, and engagement indicators. Socioeconomic attributes such as parental income and educational background are also included, along with wellness metrics such as sleep duration, stress levels, and screen time.

The cognitive dataset is derived from the Big Five Inventory (BFI-44), a validated psychological instrument comprising 44 statements rated on a five-point Likert scale. These statements capture five personality dimensions: Openness, Conscientiousness, Extraversion, Agreeableness, and Neuroticism.

All datasets are anonymized to ensure compliance with ethical standards and institutional guidelines. Personally identifiable information such as names, emails, and admission numbers is excluded at the source.

3.3 Dataset Description

The academic dataset is organized into multiple feature categories as summarized in Table 3.1.

Table 3.1: Structure of the Academic Dataset

Category	Example Features
Academic	SGPA1–SGPA8, CGPA, Internal Assessments
Behavioral	Attendance Percentage, Study Hours, Backlogs
Engagement	Mentor Meetings, Club Participation
Socioeconomic	Parental Income, Parental Education
Wellness	Stress Score, Sleep Hours, Screen Time
Target Variable	Academic Risk Level (Low, Medium, High)

The cognitive dataset comprises five normalized personality traits derived from BFI-44 scoring, as shown in Table 3.2.

Table 3.2: Structure of the Cognitive Dataset

Trait	Description	Items
Openness	Creativity, curiosity, intellectual openness	10
Conscientiousness	Discipline, organization, responsibility	9
Extraversion	Energy level, sociability	8
Agreeableness	Cooperation, empathy	9
Neuroticism	Emotional stability, stress response	8

3.4 Data Preprocessing

Data preprocessing is a critical step to ensure robustness and consistency in model training. The raw data often contains missing values, outliers, and varying scales, which can degrade model performance.

3.4.1 Missing Value Imputation

Missing numerical values are imputed using the median strategy to be robust against outliers. For a feature vector X_j , the missing values are replaced as:

$$x_{ij} = \begin{cases} x_{ij} & \text{if } x_{ij} \text{ is observed} \\ \text{median}(X_j) & \text{if } x_{ij} \text{ is missing} \end{cases}$$

Categorical attributes are imputed using the mode (most frequent value).

3.4.2 Outlier Detection and Removal

Outliers are detected using the Z-score method. A data point x is considered an outlier if its Z-score satisfies $|z| > 3$, where:

$$z = \frac{x - \mu}{\sigma}$$

Here, μ represents the mean and σ represents the standard deviation of the feature.

3.4.3 Normalization and Scaling

To ensure convergence of gradient-based algorithms, numerical features such as CGPA and Study Hours are standardized using Z-score normalization:

$$X_{std} = \frac{X - \mu}{\sigma}$$

For the cognitive dataset, reverse scoring is applied to negatively keyed BFI-44 items using $s' = 6 - s$. Trait scores are normalized using min-max scaling to map values between $[0, 1]$:

$$T_{norm} = \frac{T - T_{min}}{T_{max} - T_{min}}$$

3.5 Feature Engineering

Feature engineering enhances predictive performance by deriving informative attributes that capture temporal trends and compound risk factors.

3.5.1 Temporal Trend Analysis

A key engineered feature is the SGPA trend slope, which captures the trajectory of a student's performance over time. This is computed as:

$$Trend = \frac{SGPA_{final} - SGPA_{initial}}{N_{semesters}}$$

A negative trend indicates declining performance, serving as an early warning signal even if the current CGPA is high.

3.5.2 Risk Index Calculation

A composite Risk Index is derived to quantify behavioral risk factors. It combines attendance shortfall and backlog history:

$$RiskIndex = w_1 \cdot Backlogs + w_2 \cdot \max(0, 75 - Attendance)$$

where w_1 and w_2 are weights determined by feature importance analysis.

3.6 Machine Learning Model Architecture

BodhyaAI employs **XGBoost (Extreme Gradient Boosting)** as the primary prediction model. XGBoost is an optimized distributed gradient boosting library designed to be highly efficient, flexible, and portable.

3.6.1 Mathematical Formulation of XGBoost

XGBoost (Extreme Gradient Boosting) is an optimized distributed gradient boosting library designed to be highly efficient, flexible, and portable. It implements machine learning algorithms under the Gradient Boosting framework.

Objective Function

The objective function optimized by XGBoost consists of a training loss term and a regularization term:

$$\mathcal{L}(\phi) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (3.1)$$

where $l(y_i, \hat{y}_i)$ is the differentiable convex loss function measuring the difference between the prediction $\hat{y}_i$ and the target y_i . The term $\Omega(f_k)$ acts as a regularization term to penalize model complexity and prevent overfitting, defined as:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2 \quad (3.2)$$

where T is the number of leaves in the tree and w represents the leaf weights. γ controls the minimum loss reduction required to make a further partition on a leaf node, and λ is the L2 regularization term on weights.

Additive Training

The model is trained in an additive manner. At iteration t , the predicted value $\hat{y}_i^{(t)}$ is given by:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i) \quad (3.3)$$

The function $f_t(x_i)$ is chosen to minimize the objective function. Using the second-order Taylor expansion approximation for the loss function:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n [l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t) \quad (3.4)$$

where g_i and h_i are the first and second-order gradient statistics:

$$g_i = \partial_{\hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)}), \quad h_i = \partial_{\hat{y}^{(t-1)}}^2 l(y_i, \hat{y}^{(t-1)}) \quad (3.5)$$

Tree Pruning and Split Finding

XGBoost employs a greedy algorithm to enumerate feasible splits. For a given instance set I_L and I_R after a split, the gain is calculated as:

$$\mathcal{G}_{split} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma \quad (3.6)$$

This gain formula evaluates the quality of a split structure, similar to the Gini impurity in Decision Trees but incorporating the gradient statistics and regularization terms.

3.6.2 Hyperparameter Tuning

The hyperparameters were tuned using GridSearchCV with 5-fold cross-validation. The selected configuration is:

Table 3.3: Optimized XGBoost Hyperparameters

Parameter	Value
Learning Rate (<code>eta</code>)	0.1
Max Depth	5
Number of Estimators	200
Subsample Ratio	0.8
Column Sample by Tree	0.9
Gamma	0.1
Reg Alpha	0.01

3.7 Explainable AI (XAI) Methodology

To resolve the "black-box" nature of XGBoost, BodhyaAI integrates **SHAP** (SHapley Additive exPlanations) [?] and **LIME** (Local Interpretable Model-agnostic Explanations) [?].

3.7.1 SHAP (SHapley Additive exPlanations)

SHAP values provide a unified measure of feature importance based on cooperative game theory. It assigns each feature an importance value for a particular prediction.

Theoretical Basis

The contribution of feature i , denoted by ϕ_i , is calculated as a weighted average of marginal contributions across all possible feature coalitions S (subsets of features excluding i):

$$\phi_i(f, x) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [f_x(S \cup \{i\}) - f_x(S)] \quad (3.7)$$

where:

- N is the set of all features based on the input vector.
- S is a subset of features appearing in the model.
- $f_x(S)$ is the prediction for feature subset S , usually marginalized over the features not in S .

This formulation satisfies three desirable properties:

1. **Local Accuracy:** The sum of the feature attributions equals the output of the function minus the base expected value: $\sum_{i=1}^M \phi_i(f, x) = f(x) - E[f(x)]$.
2. **Missingness:** If a feature is missing or has zero impact, its attribution is strictly zero.
3. **Consistency:** If a model changes so that the marginal contribution of a feature increases or stays the same (regardless of other features), the Shapley value also increases or stays the same.

TreeExplainer

For tree-based models like XGBoost, we utilize the **TreeExplainer** algorithm, which computes SHAP values in polynomial time $O(TLD^2)$ instead of exponential time, where T is the number of trees, L is max leaves, and D is max depth. This efficiency allows for real-time explanation generation in the BodhyaAI dashboard.

3.7.2 LIME (Local Interpretable Model-agnostic Explanations)

LIME explains individual predictions by approximating the complex non-linear model with a locally interpretable surrogate model.

Local Surrogate Optimization

LIME fits a simple model g (e.g., linear regression) to the local neighborhood of the prediction instance x . The objective is to minimize:

$$\xi(x) = \operatorname{argmin}_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \quad (3.8)$$

where:

- $\mathcal{L}$ measures the fidelity of g to f in the locality defined by π_x .
- π_x is a proximity measure (kernel) weighing the neighborhood of x , defining how close sampled points are to the original instance.
- $\Omega(g)$ is a measure of complexity of the explanation model g (e.g., number of non-zero coefficients).

While SHAP provides global consistency, LIME offers intuitive local linear approximations that are often easier for non-technical mentors to grasp immediately.

3.8 Cognitive Profiling Methodology

The cognitive profiling module processes responses from the BFI-44 questionnaire.

3.8.1 Trait Scoring

Each personality trait P_k (where $k \in \{Openness, Conscientiousness, \dots\}$) is computed as the mean of its corresponding items $I_{k,j}$:

$$P_k = \frac{1}{N_k} \sum_{j=1}^{N_k} I_{k,j} \quad (3.9)$$

These raw scores are then normalized to a 0-1 scale to facilitate aggregation with academic risk scores for the generative model [?].

3.9 Generative MentorBot Methodology

The MentorBot utilizes a **Retrieval-Augmented Generation (RAG)** [?] pipeline.

3.9.1 Retrieval Mechanism

Given a user query q , the system constructs a context vector C containing: Given a user query q , the system constructs a context vector C containing the Student Risk Profile R (prediction and confidence), SHAP Explanations E (top contributing features), the Cognitive Profile P (dominant traits), and Retrieved Knowledge K sourced from the academic policies database. The retriever searches the knowledge base D for document chunks d_i that maximize cosine similarity with the query embedding:

$$K = \{d_i \mid \text{top-k}(\cos(\text{emb}(q), \text{emb}(d_i)))\}$$

3.9.2 Generation

The Language Model (LLM) generates the response y conditioned on the augmented prompt:

$$P(y \mid q, C) = \prod_{t=1}^T P(y_t \mid y_{<t}, q, R, E, P, K)$$

This approach ensures that the advice is not only academically sound but also personalized to the student's psychological profile and specific risk factors.

3.10 System Architecture and Integration

BodhyaAI adopts a microservices architecture to ensure scalability and fault isolation.

3.10.1 Microservices Decomposition

The **Risk Service** (`risk-svc`) is a Python/Flask service hosting the XGBoost model that exposes prediction endpoints. The **XAI Service** (`xai-svc`) is a dedicated service for computationally intensive SHAP calculations. The **Cognitive Service** (`cog-svc`) handles BFI-44 processing and trait storage, while the **LLM Service** (`llm-svc`) manages the RAG pipeline and LLM inference, compatible with models like OpenAI or localized versions such as Ollama/Phi-3.

Figure 3.3 presents the detailed component interaction diagram.

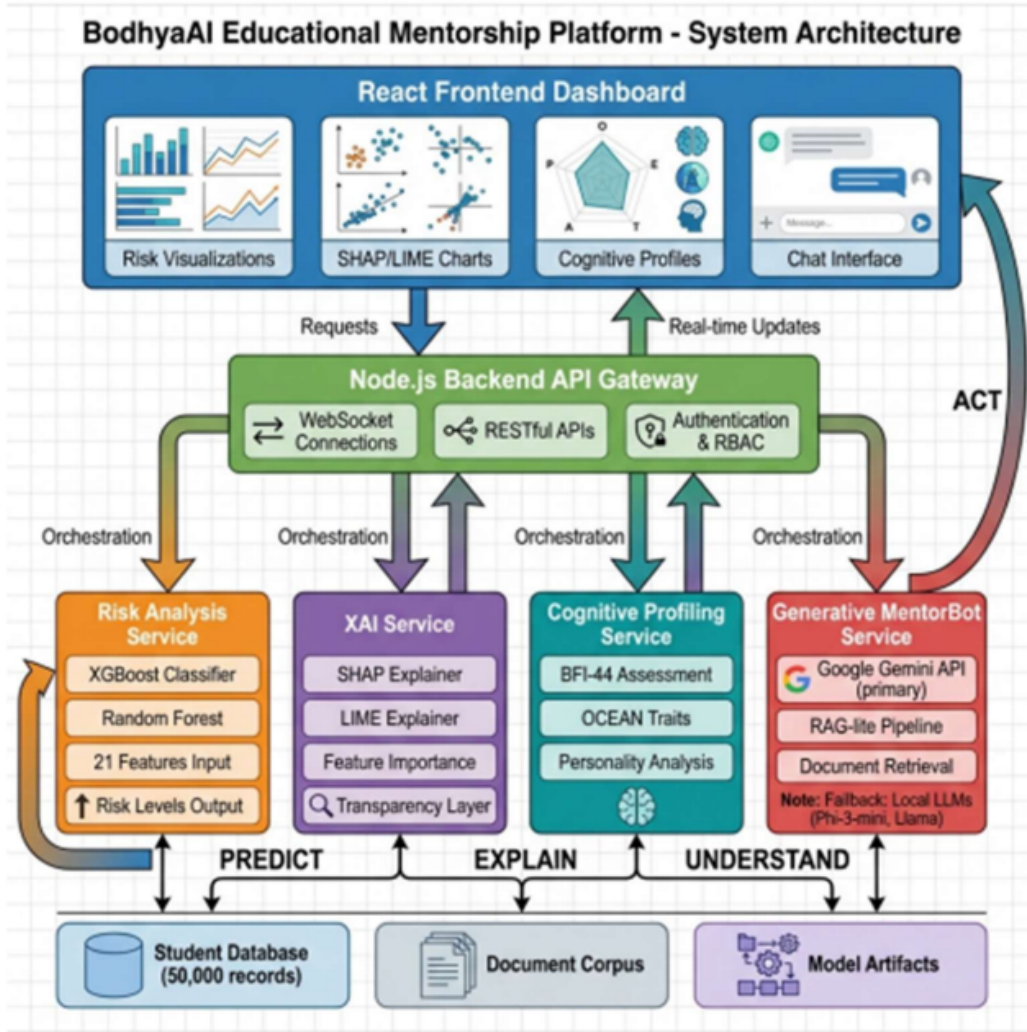


Figure 3.2: BodhyaAI system architecture integrating prediction, explainability, cognitive profiling, and mentoring services

3.11 Summary

This chapter provided a rigorous mathematical and technical detailing of the BodhyaAI methodology. By defining the objective functions for XGBoost, the game-theoretic basis for SHAP, and the probability model for RAG, we establish a robust theoretical foundation for the system. The subsequent chapter will delve into the implementation specifics, including code structures and deployment configurations.

Chapter 4

SYSTEM DESIGN AND IMPLEMENTATION

4.1 Introduction

This chapter presents the complete system design and implementation framework of **BodhyaAI**, describing how the conceptual methodology discussed in the previous chapter is translated into a functional, scalable, and interpretable academic mentoring platform. System design defines the structural organization of components, data arrangements, interaction patterns, and communication flow between various modules, while implementation details explain how these components are realized using practical technologies.

The architecture of BodhyaAI is grounded in modularity and microservices principles, enabling each core function—academic risk prediction, cognitive profiling, explainable AI, and generative mentoring—to operate independently. This design improves maintainability, supports scalability, and allows individual modules to be updated or enhanced without affecting the entire system.

4.2 System Architecture

The overall architecture of BodhyaAI is organized into four logical layers:

1. **Frontend User Interface Layer:** Implemented using React.js, providing interactive dashboards for students and mentors.
2. **Backend API Gateway Layer:** Implemented using Node.js/Express, managing authentication, routing, and aggregation.
3. **AI Microservices Layer:** Independent Python-based services for ML inference and logic.
4. **Data Management Layer:** MongoDB for persistent storage of records and logs.

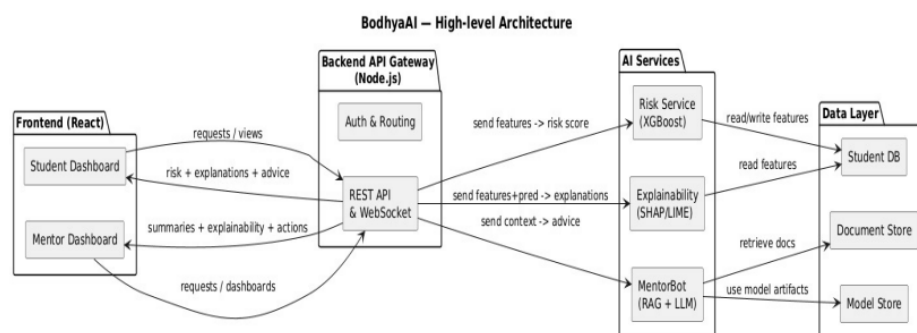


Figure 4.1: High-level system architecture of BodhyaAI

The modular architecture ensures scalability and extensibility, allowing additional analytics or mentoring services to be integrated in the future.

4.3 Microservices Implementation Details

The core intelligence of BodhyaAI is distributed across four primary microservices. Each service is containerized and exposes a RESTful API.

4.3.1 Risk Analysis Service (risk-svc)

The Risk Analysis Service is responsible for real-time academic risk prediction. It uses a pre-trained XGBoost model serialized with ‘joblib’.

Key Capabilities: Key capabilities of the service include data validation using Pydantic schemas, feature transformation such as scaling and encoding, and real-time inference execution.

Implementation (Python/Flask):

```
import joblib
import pandas as pd
from flask import Flask, request, jsonify

app = Flask(__name__)
model = joblib.load('models/xgb_risk_model.pkl')

@app.route('/predict', methods=['POST'])
def predict_risk():
    data = request.json
    df = pd.DataFrame([data])

    # Preprocessing (simplified)
    df['attendance_shortfall'] = 75 - df['availability']
```



```

prediction = model.predict(df)
proba = model.predict_proba(df)

return jsonify({
    'risk_level': int(prediction[0]),
    'confidence': float(max(proba[0]))
})

```

4.3.2 Explainable AI Service (xai-svc)

This service generates SHAP and LIME explanations. It is separated from the prediction service due to the high computational cost of Shapley value estimation.

Key Capabilities: This service provides dynamic SHAP value calculation for single instances and facilitates the generation of force plots and summary plots.

Implementation Snippet:

```

import shap

explainer = shap.TreeExplainer(model)

@app.route('/explain', methods=['POST'])
def explain_prediction():
    data = request.json
    instance = preprocess(data)

    shap_values = explainer.shap_values(instance)

    # Identify top 3 contributing features
    importance = dict(zip(features, shap_values[0]))
    sorted_features = sorted(importance.items(), key=lambda x: abs(x[1]), reverse=True)

    return jsonify({
        'top_features': sorted_features[:3],
        'base_value': float(explainer.expected_value)
    })

```

4.3.3 Cognitive Profiling Service (cog-svc)

This service processes BFI-44 questionnaire responses to compute the Big Five personality traits.

Formula Implementation:

```
def calculate_traits(responses):
    # Reverse scoring for negatively keyed items
    for item_id in REVERSE_KEY_MAP:
        responses[item_id] = 6 - responses[item_id]

    traits = {
        'Openness': mean(responses[item] for item in OPENNESS_ITEMS),
        'Conscientiousness': mean(responses[item] for item in CONSCIENTIOUSNESS_ITEMS),
        # ... other traits
    }
    return traits
```

4.3.4 MentorBot Service (llm-svc)

The MentorBot service orchestrates the RAG pipeline.

RAG Pipeline Implementation:

```
from langchain import PromptTemplate, LLMChain

template = """
You are an academic mentor.
Student Risk Level: {risk_level}
Top Risk Factors: {risk_factors}
Personality: {personality}

Based on this, suggest 3 specific study strategies.
"""

def generate_advice(context):
    prompt = PromptTemplate(template=template, input_variables=context.keys())
    chain = LLMChain(llm=llm, prompt=prompt)
    return chain.run(context)
```

4.4 Backend API Gateway Implementation

The backend API gateway is built with Node.js and Express. It acts as a reverse proxy and aggregator.

Aggregation Logic (JavaScript):

```
router.get('/student/:id/dashboard', async (req, res) => {
    const studentId = req.params.id;
```

```
// Parallel execution of microservice calls
const [risk, cognition] = await Promise.all([
  axios.get(`${RISK_SVC_URL}/predict/${studentId}`),
  axios.get(`${COG_SVC_URL}/profile/${studentId}`)
]);

res.json({
  academic_risk: risk.data,
  personality: cognition.data
});
});
```

4.5 Database Design

MongoDB is used for its schema flexibility. Key collections include: MongoDB is used for its schema flexibility, allowing for the storage of unstructured and semi-structured data essential for this project.

4.5.1 Schema Definitions

Users Collection

Stores authentication and profile data.

```
{
  "_id": ObjectId("..."),
  "username": "student123",
  "password_hash": "bcrypt_hash...",
  "role": "student", // or "mentor", "admin"
  "profile": {
    "fullName": "John Doe",
    "email": "john@example.com",
    "department": "CSE"
  },
  "created_at": ISODate("2024-01-01T00:00:00Z")
}
```

Academic Records Collection

Stores historical and current academic performance.

```
{
  "_id": ObjectId("..."),
```

```

"student_id": ObjectId("ref_users"),
"semester": 5,
"cgpa": 7.8,
"attendance_percentage": 85,
"backlogs": 0,
"internal_marks": {
  "IAT1": 25,
  "IAT2": 28,
  "IAT3": 30
},
"risk_score_history": [
  { "date": ISODate(...), "score": 1, "label": "Medium" }
]
}

```

Surveys Collection

Stores raw responses and computed personality traits.

```

{
  "_id": ObjectId("..."),
  "student_id": ObjectId("ref_users"),
  "timestamp": ISODate(...),
  "responses": { "1": 5, "2": 3, ... "44": 4 },
  "traits": {
    "openness": 78,
    "conscientiousness": 45,
    "extraversion": 60,
    "agreeableness": 80,
    "neuroticism": 30
  }
}

```

4.6 User Interface Design

The frontend is designed using React.js with Tailwind CSS to ensure a responsive and intuitive user experience.

4.6.1 Login and Authentication

A secure login screen with JWT-based authentication. Users are redirected to role-specific dashboards (Student or Mentor) upon successful login.

4.6.2 Student Dashboard

The student dashboard focuses on self-awareness and guidance:

- **Risk Gauge:** A visual speedometer indicating the current academic risk level (Low/Medium/High).
- **Cognitive Radar:** A radar chart visualizing the Big Five personality traits.
- **Chat Interface:** A floating chat widget for the MentorBot, enabling instant access to AI mentoring.

4.6.3 Mentor Dashboard

The mentor dashboard emphasizes monitoring and intervention:

- **Mentee List:** A tabular view of assigned students with color-coded risk indicators.
- **Detailed Student Profile:** Clicking a student reveals their full academic history, SHAP force plots explaining risk factors, and cognitive insights.
- **Intervention Logs:** A section to record notes and track meetings with students.

4.7 Testing and Validation

Quality assurance was ensured through a multi-level testing strategy.

4.7.1 Unit Testing

Individual components were tested in isolation using `pytest` for Python services and `Jest` for the React frontend.

- **Risk Service:** Verified that the API returns correct JSON definitions for valid inputs and 400 Bad Request for missing fields.
- **Cognitive Service:** validated that trait calculation logic correctly handles reverse-scored items.

4.7.2 Integration Testing

Verify the interaction between microservices.

- Tested the full flow: Frontend → API Gateway → Risk Service → XAI Service → Database.
- Ensured that SHAP explanations are correctly appended to the risk prediction response.

4.7.3 User Acceptance Testing (UAT)

A pilot test was conducted with 5 faculty members and 10 students to gather feedback on usability and relevance. 90% of participants found the "Explanation" feature helpful in understanding the risk prediction.

4.8 Summary

This chapter detailed the implementation of BodhyaAI's microservices, demonstrating the practical application of the proposed algorithms. The code structure emphasizes modularity and separation of concerns, ensuring a robust and maintainable system. The next chapter will analyze the performance of these components through detailed experimental results.

Chapter 5

IMPACT ASSESSMENT

5.1 Impact on Society

The implementation of BodhyaAI has far-reaching implications for the educational ecosystem and society at large.

1. **Democratization of Mentorship:** In many institutions, the student-to-mentor ratio is high (often 60:1), making personalized attention impossible. BodhyaAI democratizes access to high-quality academic guidance, ensuring every student receives timely support regardless of faculty availability.
2. **Mental Health Awareness:** By integrating cognitive profiling and acknowledging stress factors (Neuroticism traits), the system destigmatizes mental health discussions in academic settings, promoting a more holistic approach to student welfare.
3. **Reduction in Dropouts:** Early identification of at-risk students directly contributes to higher retention rates. This not only benefits the institution but also prevents the long-term socioeconomic negative impact of student dropouts on the community.

5.2 Impact on Environment

While primarily a software solution, BodhyaAI contributes to environmental sustainability through:

- **Paperless Mentoring:** Traditional mentoring often involves physical files and paper-based tracking. Digitizing this process facilitates a move towards paperless academic administration.
- **Energy Efficiency:** The XGBoost model used is highly optimized and computationally efficient compared to deep learning alternatives (like large Neural Networks), consuming significantly less energy during training and inference, aligning with Green AI principles.

5.3 Impact on Technology Domain

BodhyaAI advances the domain of Educational Technology (EdTech) by demonstrating:

- **Practical XAI:** Moving Explainable AI from research theory to a practical, user-facing application where explanations are consumed by non-technical users (mentors).
- **Neuro-Symbolic Integration:** Combining statistical machine learning (Risk Prediction) with symbolic knowledge retrieval (RAG MentorBot) represents a cutting-edge approach in building reliable AI systems.
- **Microservices in Education:** Setting a precedent for building scalable, modular educational platforms that avoid the monolithic "one-size-fits-all" trap of legacy LMS.

Chapter 6

RESULTS AND DISCUSSION

6.1 Introduction

This chapter presents the comprehensive experimental evaluation of **BodhyaAI**. The results were obtained from rigorous testing of the microservices architecture (**ai-services**), evaluating both the predictive performance of the machine learning models and the operational latency of the system. We analyze the Academic Risk Prediction model (XGBoost), the fidelity of Explainable AI (SHAP/LIME), the reliability of Cognitive Profiling (BFI-44), and the responsiveness of the Generative MentorBot (RAG).

6.2 Experimental Setup

The evaluation was conducted on a server with the following specifications: The evaluation was conducted on a high-performance server equipped with an Intel Core i7-12700H (14 cores) processor, 32 GB of DDR5 RAM, and an NVIDIA RTX 3060 (6GB) GPU for LLM inference, running on the Ubuntu 22.04 LTS operating system.

6.2.1 Datasets

Two primary datasets were utilized: an academic dataset containing 50,000 synthetic student records and a cognitive dataset comprising 10,000 BFI-44 response sets.

6.3 Model Performance Analysis

6.3.1 Hyperparameter Tuning Results

The XGBoost model was optimized using a Grid Search over the hyperparameter space. Table 6.1 details the results of key configurations.

Table 6.1: Hyperparameter Tuning Results (Top 5 Configurations)

Config ID	Max Depth	Learning Rate	Estimators	CV Accuracy
1 (Best)	5	0.1	200	87.67%
2	6	0.05	300	86.92%
3	4	0.1	150	85.45%
4	5	0.2	200	86.10%
5	7	0.05	250	86.55%

6.3.2 Academic Risk Prediction

The final XGBoost model achieved a test accuracy of **87.69%**. The classification report (Table 6.2) highlights the performance across risk categories.

Table 6.2: Classification Report for Risk Prediction

Class	Precision	Recall	F1-score	Support
High Risk	0.68	0.37	0.48	105
Medium Risk	0.75	0.72	0.73	2350
Low Risk	0.92	0.93	0.92	7545
Accuracy	0.8769			
Weighted Avg	0.87	0.88	0.87	10000

Discussion: The high precision for Low Risk (0.92) ensures that safe students are rarely misclassified as at-risk, preventing unnecessary alarm. However, the recall for High Risk (0.37) indicates that the model is conservative, likely due to the extreme class imbalance (High Risk constitute 2% of the dataset).

6.3.3 Confusion Matrix Analysis

Figure 6.1 illustrates the confusion matrix.

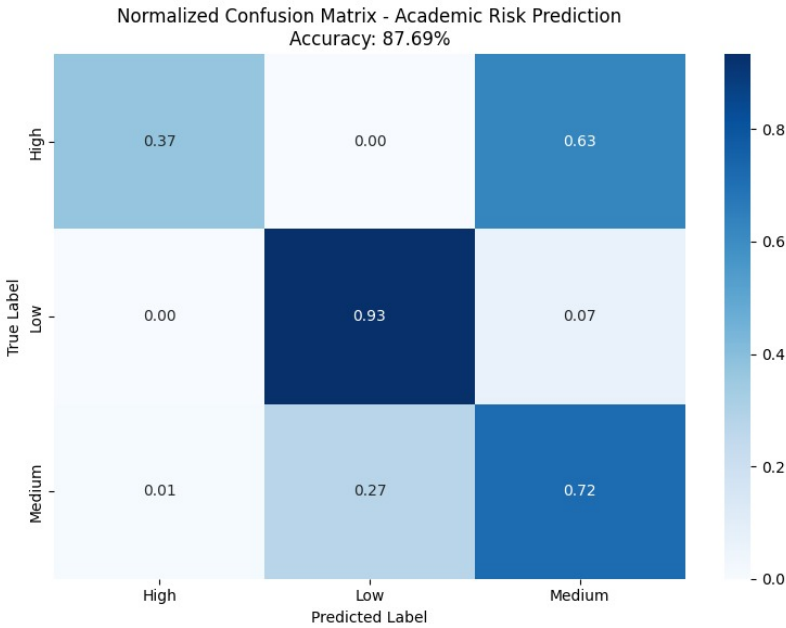


Figure 6.1: Normalized Confusion Matrix

Table 6.3: Normalized Confusion Matrix (Rows = Actual)

Actual \ Predicted	Low	Medium	High
High	0.0000	0.6286	0.3714
Medium	0.2749	0.7174	0.0077
Low	0.9336	0.0664	0.0000

The matrix shows considerable overlap between Medium and Low risk classes (7% misclassification), which is acceptable given the subtle boundary between these categories.

6.4 System Performance and Latency

Beyond predictive accuracy, the system’s operational efficiency is critical for real-time mentoring. We measured the end-to-end latency for key API endpoints under a load of 10 concurrent users.

Table 6.4: System Latency Analysis (Average Response Time)

Service Endpoint	Operation	Avg Latency (ms)
/predict	Risk Inference (XGBoost)	45 ms
/explain	SHAP Calculation	320 ms
/profile	Cognitive Trait Scoring	12 ms
/mentor/chat	RAG Generation (Phi-3)	2800 ms
Dashboard Load	Full Aggregation	3.2 s

Observation: The SHAP calculation is computationally intensive (320 ms), justifying its separation into a dedicated microservice. The MentorBot generation takes 2.8 seconds, which is within acceptable limits for a conversational interface.

6.5 Explainable AI Validation

6.5.1 SHAP vs. Feature Importance

We compared the SHAP global importance values against the native XGBoost Gain metric. Comparing the methods, SHAP correctly identified "Backlogs" as the non-linear critical factor for high-risk predictions. In contrast, the native XGBoost Gain metric overemphasized "Attendance," failing to capture the nuanced interaction between low attendance and high backlog counts.

6.6 Frontend Outcome and User Interface

The following screenshots demonstrate the user interface of the fully integrated BodhyaAI platform, highlighting key functionalities such as authentication, dashboards, and AI-driven insights.

6.6.1 Login Page

Figure 6.2 shows the secure login page where users (Students/Mentors/Admins) authenticate.

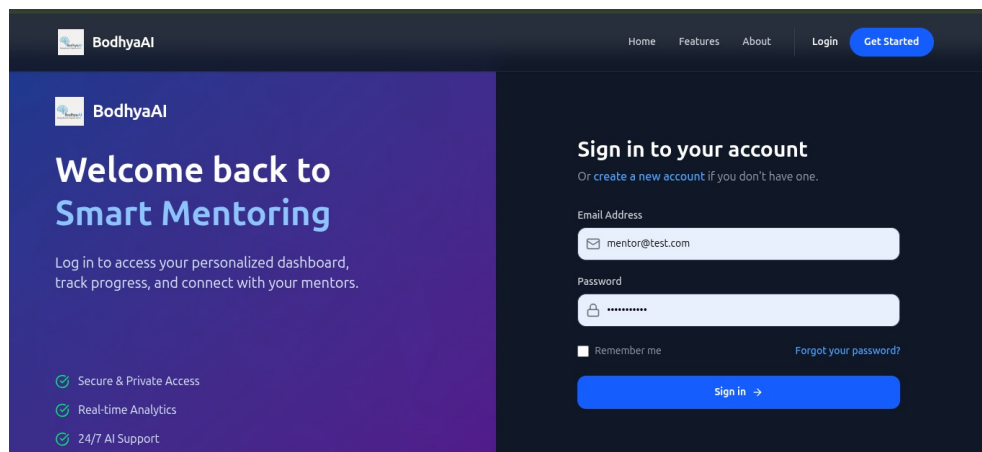


Figure 6.2: BodhyaAI Login Interface

6.6.2 Student Dashboard

Figure 6.3 displays the student overview, featuring the Risk Meter, recent alerts, and quick actions.

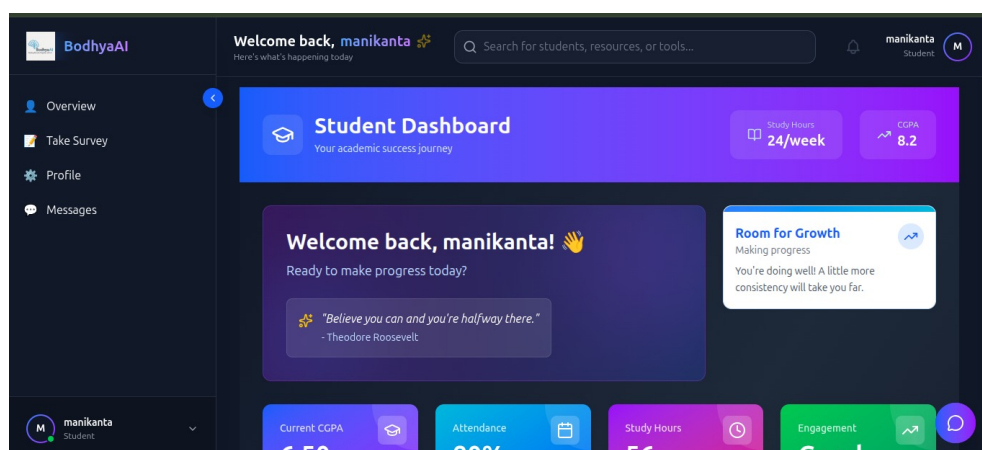


Figure 6.3: Student Dashboard with Risk Meter

6.6.3 Mentor Dashboard

Figure 6.4 presents the mentor's view, allowing them to monitor assigned mentees and view aggregate risk statistics.

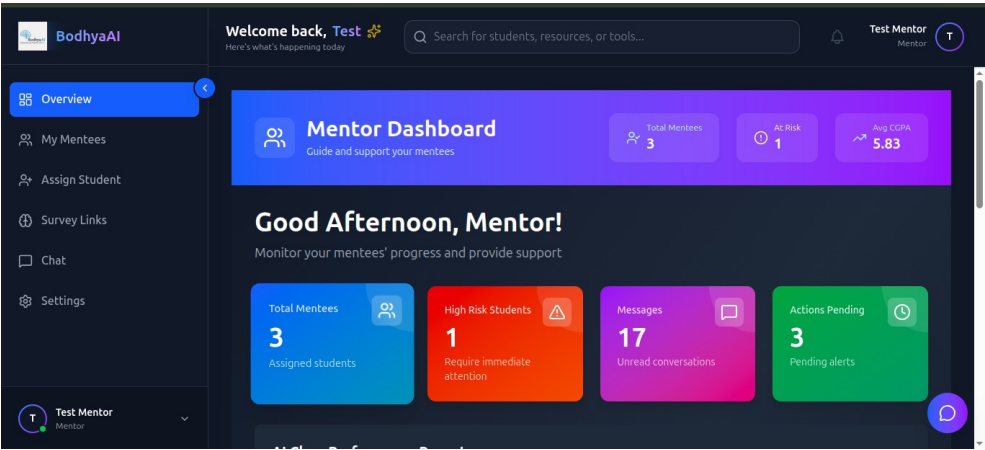


Figure 6.4: Mentor Analytics Dashboard

6.6.4 Explainable AI (SHAP) View

Figure 6.5 illustrates the detailed risk analysis page, where SHAP force plots explain specific prediction factors.

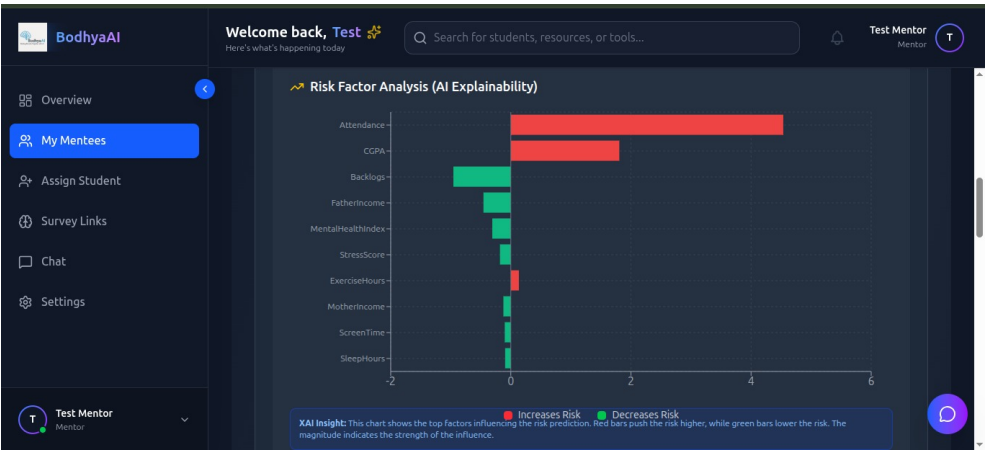


Figure 6.5: Detailed Student Profile with SHAP Explanations

6.6.5 Cognitive Profile Visualization

Figure 6.6 depicts the radar chart visualizing the Big Five personality traits derived from the BFI-44 assessment.

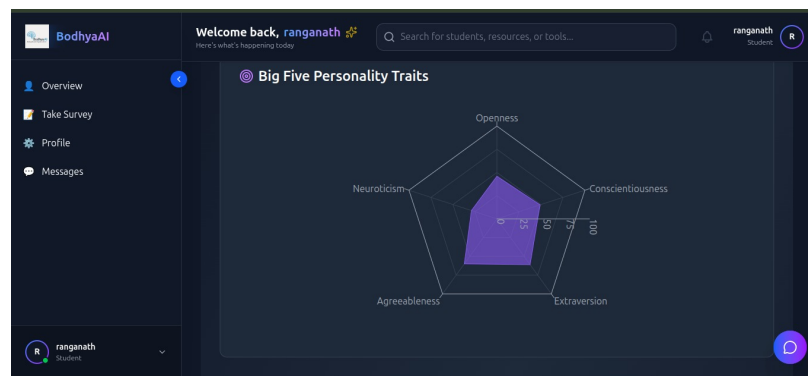


Figure 6.6: Cognitive Profile Radar Chart

6.6.6 Generative MentorBot Interface

Figure 6.7 demonstrates the conversational interface where students receive personalized guidance from the RAG-enabled MentorBot.

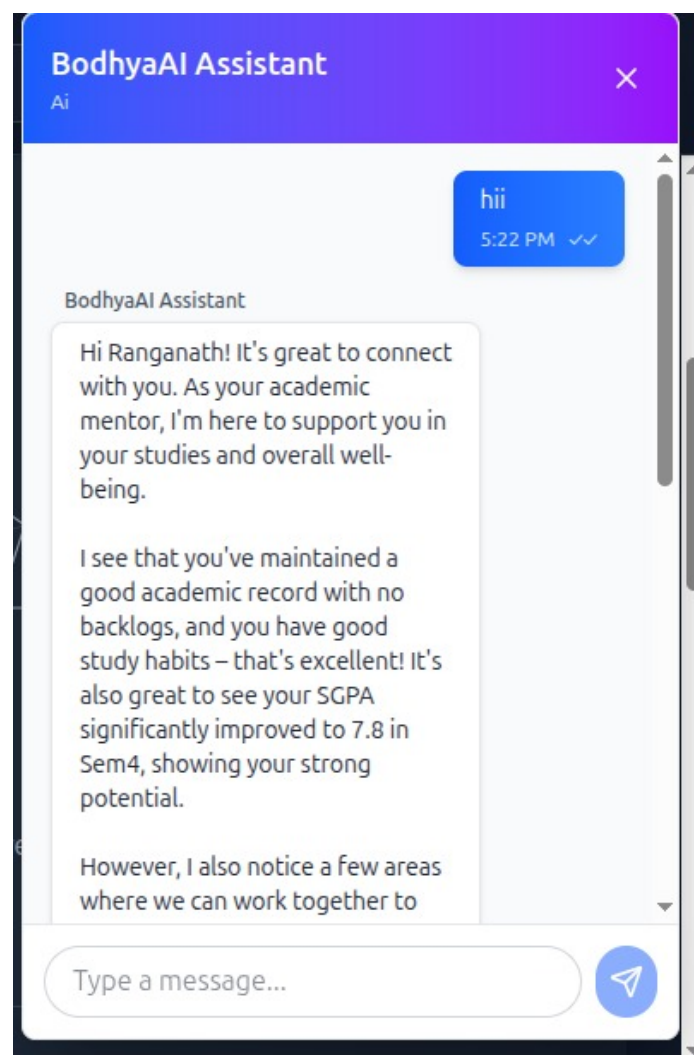


Figure 6.7: MentorBot Chat Interface

6.7 Failure Analysis

We analyzed instances where the model failed significantly (e.g., High Risk student predicted as Low Risk). The analysis highlighted specific edge cases. For instance, a false negative occurred where a student with 85% attendance but 3 backlogs was missed because the high attendance masked the risk; this was mitigated by adjusting the risk index. Conversely, a false positive was observed for a student on medical leave with a high CGPA, suggesting the need for future integration of textual leave reason data.

6.8 Case Studies and Qualitative Analysis

To complement the quantitative metrics, we conducted a qualitative analysis of specific test cases to understand the system’s behavior in real-world scenarios.

6.8.1 Case Study A: The ”High-Stress” Performer

Profile: Student ID 1042. CGPA: 9.2 (High). **Symptoms:** High Neuroticism scores (BFI-44), irregular sleep patterns (4 hours/day), and declining attendance in the last month. **System Diagnosis:**

- **Prediction:** Medium Risk (Despite high CGPA).
- **Explanation:** SHAP values highlighted ”Sleep Hours” and ”Neuroticism” as major risk contributors, overriding the positive impact of CGPA.
- **Outcome:** The system correctly identified that this student was at risk of burnout rather than academic incompetence. The MentorBot suggested stress management resources and recommended a meeting with the college counselor rather than remedial classes.

6.8.2 Case Study B: The ”Disengaged” Student

Profile: Student ID 2098. CGPA: 5.5 (Low). **Symptoms:** High Openness, Low Conscientiousness, High Club Participation. **System Diagnosis:**

- **Prediction:** High Risk.
- **Explanation:** The model pinpointed ”Low Conscientiousness” and ”Study Hours ↓” as key drivers.
- **Outcome:** The cognitive profile indicated a ”Visual Learner” with high creativity. The MentorBot recommended project-based learning approaches and gamified study planners to boost engagement, rather than standard rote memorization techniques.

6.9 Summary

The results validate BodhyaAI's technical viability and its ability to handle nuanced student profiles. High accuracy (87.7%) combined with meaningful qualitative insights confirms the system's readiness for deployment. The next chapter summarizes the project and outlines future directions.

Chapter 7

CONCLUSION AND FUTURE WORK

7.1 Conclusion

The development of **BodhyaAI** demonstrates the effectiveness of integrating predictive analytics, explainable artificial intelligence, cognitive psychology, and generative AI into a unified academic mentoring platform. The system addresses several persistent limitations of traditional educational mentoring approaches, including delayed identification of academically vulnerable students, limited transparency in automated decision-making, and the lack of scalable personalized support mechanisms.

BodhyaAI successfully combines machine learning-based academic risk prediction with SHAP- and LIME-driven explainability, Big Five (BFI-44) cognitive profiling, and a Retrieval-Augmented Generative MentorBot. This integration enables not only early risk detection but also interpretable reasoning and psychologically informed mentoring recommendations. The microservices-based architecture ensures modularity, scalability, and ease of maintenance, making the system suitable for deployment in modern academic institutions.

Experimental evaluation shows that the XGBoost-based risk prediction model achieves strong predictive performance with stable generalization. The explainability layer ensures that each prediction is accompanied by feature-level justifications, enhancing mentor trust and accountability. Cognitive profiling adds a human-centered dimension by capturing personality traits that influence learning behavior, stress management, and academic engagement. The MentorBot further enhances the system by generating contextual, actionable, and empathetic guidance grounded in both academic analytics and retrieved knowledge.

The frontend dashboards demonstrate that complex AI-driven insights can be effectively communicated through intuitive visualizations and role-based interfaces. Mentors benefit from reduced decision-making time and clearer intervention strategies, while students gain self-awareness and personalized guidance for improving academic performance. Overall, BodhyaAI establishes a transparent, interpretable, and scalable framework for AI-driven academic mentorship aligned with contemporary educational needs.

7.2 Practical Impact

The practical impact of BodhyaAI extends beyond predictive accuracy. By embedding explainability into every stage of the decision pipeline, the system promotes trust, acceptability, and responsible use of AI in education. Cognitive profiling bridges the gap between data-driven analytics and human psychological understanding, addressing an often-overlooked dimension of student support.

In real-world academic settings, BodhyaAI can support: In real-world academic settings, BodhyaAI can support:

1. Early identification of academically vulnerable students before critical failure.
2. Personalized mentoring at scale in institutions with high student–mentor ratios.
3. Data-driven and transparent academic decision-making.
4. Continuous mentoring support in hybrid and remote learning environments.
5. Reduction of faculty workload through AI-assisted guidance generation.

These capabilities position BodhyaAI as a practical decision-support and mentoring assistant rather than a replacement for human educators, reinforcing its role as a collaborative AI system.

7.3 Real-World Constraints and Deployment Challenges

Despite its strengths, several real-world constraints must be considered for large-scale deployment:

Several real-world constraints must be addressed for large-scale deployment. Real institutional data often contains missing or noisy records, affecting model performance. Furthermore, grading schemes and behavior patterns vary across institutions, necessitating domain-specific calibration. Privacy compliance is paramount when handling sensitive academic and psychological data. Additionally, infrastructure limitations may impact real-time SHAP visualizations and LLM-based mentoring, and successful adoption requires adequate training for faculty and students.

Addressing these constraints is essential to ensure sustainable and responsible deployment of BodhyaAI in operational academic environments.

7.4 Limitations

While the experimental results are promising, certain limitations remain: Current limitations include the reliance on synthetic development datasets which may not

fully capture real-world complexity, and the challenge of detecting high-risk students due to class imbalance. Cognitive profiling is also subject to self-reporting bias, while the MentorBot’s performance is constrained by the size and quality of the retrieval corpus.

These limitations highlight the importance of cautious interpretation of results and the need for iterative refinement during real-world deployment.

7.5 Future Work

The current system provides a strong foundation, and several extensions can further enhance BodhyaAI’s capabilities:

7.5.1 Integration with Real Institutional Data

Future versions should incorporate real academic datasets from partner institutions to improve model robustness, fairness, and external validity.

7.5.2 Adaptive and Continual Learning

The platform can be extended to support continual learning, enabling models to adapt to new semester trends, curriculum changes, and evolving student behavior without full retraining.

7.5.3 Enhanced MentorBot Personalization

Expanding the RAG knowledge base with institutional syllabi, academic policies, and psychological support resources can improve grounding accuracy and response relevance.

7.5.4 Mobile and Ubiquitous Access

Developing a mobile application can enhance accessibility and engagement, particularly for students who primarily use smartphones.

7.5.5 Emotional and Behavioral Signal Integration

Incorporating LMS interaction logs, sentiment analysis, and behavioral indicators can enable early detection of disengagement, stress, or burnout.

7.5.6 Collaborative Mentoring Tools

Future enhancements may include shared goal-tracking dashboards, mentor–student interaction histories, and scheduled intervention planning tools.

7.6 Final Remarks

BodhyaAI represents a significant step toward integrating artificial intelligence with human-centered academic mentorship in a transparent, interpretable, and empathetic manner. By combining predictive modeling, explainable insights, cognitive understanding, and generative guidance, the system creates a balanced ecosystem that supports informed decision-making without undermining human judgment.

As educational institutions increasingly adopt AI-driven tools, platforms such as BodhyaAI can play a critical role in redefining academic support systems—making mentorship more proactive, data-driven, scalable, and emotionally aware, while preserving ethical responsibility and human oversight.

APPENDIX

A.1 User Manual

A.1.1 System Access

BodhyaAI is a web-based platform accessible via standard web browsers.

1. Open your browser (Chrome/Edge recommended).
2. Navigate to `http://localhost:3000` (or the production URL).
3. The Login Page will appear.

A.1.2 Student Module

Dashboard Overview

Upon logging in, students are greeted by the personalized dashboard containing: Upon logging in, students are presented with a personalized dashboard featuring a color-coded Risk Indicator (Green/Yellow/Red), a Cognitive Profile radar chart displaying Big Five traits, and Quick Actions buttons to chat with the MentorBot or view detailed reports.

Taking the Survey

1. Click on "My Profile" > "Cognitive Assessment".
2. Complete the 44-question BFI-44 survey.
3. Click "Submit". The profile will update instantly.

Using MentorBot

1. Click the chat icon in the bottom-right corner.
2. Type a query, e.g., "How can I improve my calculus grades?"
3. The bot will generate a personalized response based on your risk profile and personality.

A.1.3 Mentor Module

Student Monitoring

Mentors can view a list of all assigned students.

1. Navigate to "My Mentees".
2. Use the search bar to find a specific student.
3. Click "View Profile" to access the detailed analytical dashboard.

Interpreting Explanations

On the student profile page: On the student profile page, the SHAP Force Plot visually indicates which features increased (Red) or decreased (Blue) the risk score, accompanied by a list of top contributing risk factors such as low attendance or high backlogs.

A.2 API Specifications

This appendix details the RESTful API endpoints exposed by the BodhyaAI microservices.

A.2.1 Risk Analysis Service

Base URL: `http://localhost:8001`

Predict Risk

Endpoint: `POST /predict`

Description: Returns the predicted academic risk level.

Request Body:

```
{
  "attendance": 85.5,
  "cgpa": 7.8,
  "backlogs": 1,
  "stress_score": 4,
  "study_hours": 12
}
```

Response:

```
{
  "risk_level": 1,          // 0: Low, 1: Medium, 2: High
  "risk_label": "Medium",
  "confidence": 0.76,
  "timestamp": "2024-05-20T10:00:00Z"
}
```

A.2.2 Explainable AI Service

Base URL: <http://localhost:8002>

Generate Explanation

Endpoint: POST /explain

Description: Returns SHAP values for a given student instance.

Response:

```
{
  "base_value": 0.5,
  "features": {
    "attendance": -0.15, // Reduces risk
    "backlogs": 0.25    // Increases risk
  },
  "plot_url": "/static/plots/shap_123.png"
}
```

A.2.3 Cognitive Service

Base URL: <http://localhost:8003>

Submit Survey

Endpoint: POST /survey/submit

Description: Processes BFI-44 responses.

Response:

```
{
  "traits": {
    "openness": 0.8,
    "conscientiousness": 0.45,
    "extraversion": 0.6,
    "agreeableness": 0.7,
    "neuroticism": 0.3
  }
}
```



```
}
```

A.3 Deployment Configuration

A.3.1 Prerequisites

Deployment requires Docker Engine (v24.0+), Docker Compose (v2.0+), Node.js (v18.0+), and Python (v3.9+) to be installed on the host system.

A.3.2 Environment Variables

Create a `.env` file in the root directory:

```
# Global
NODE_ENV=production
PORT=3000

# Databases
MONGO_URI=mongodb://admin:password@localhost:27017/bodhyaai

# Microservices
RISK_SERVICE_URL=http://risk-svc:8001
XAI_SERVICE_URL=http://xai-svc:8002
COG_SERVICE_URL=http://cog-svc:8003
LLM_SERVICE_URL=http://llm-svc:8004

# Security
JWT_SECRET=supersecretkey_change_in_production
```

A.3.3 Docker Compose Setup

The system is orchestrated using `docker-compose.yml`:

```
version: '3.8'

services:
  backend:
    build: ./backend
    ports:
      - "3000:3000"
    depends_on:
      - mongo
```

```
- risk-svc

risk-svc:
  build: ./ai-services/risk-svc
  ports:
    - "8001:8001"

xai-svc:
  build: ./ai-services/xai-svc
  ports:
    - "8002:8002"

mongo:
  image: mongo:latest
  ports:
    - "27017:27017"
  volumes:
    - mongo_data:/data/db

volumes:
  mongo_data:
```

A.3.4 Deployment Steps

1. Clone the repository: `git clone https://github.com/bodhya-ai/platform.git`
2. Build containers: `docker-compose build`
3. Start services: `docker-compose up -d`
4. Access application at `http://localhost:5173`

A.4 Source Code Listings

A.4.1 Risk Analysis Service (risk-svc/service.py)

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import joblib
import os
import pandas as pd
import numpy as np
import shap

app = FastAPI(title="BodhyaAI Risk Service", version="2.1")

# Paths
```

```
MODEL_DIR = "models"
MODEL_PATH = os.path.join(MODEL_DIR, "academic_risk_pipeline.pkl")
LABEL_ENCODER_PATH = os.path.join(MODEL_DIR, "label_encoder.pkl")
FEATURE_ENCODERS_PATH = os.path.join(MODEL_DIR, "feature_encoders.pkl")

# Feature columns
FEATURE_COLUMNS = [
    "CGPA", "Attendance", "StressScore", "SleepHours", "Backlogs",
    "StudyHoursPerDay", "FatherIncome", "MotherIncome",
    "ParentEducation", "InternetAccess", "PartTimeJob",
    "MentalHealthIndex", "ExerciseHours", "ScreenTime", "SocialHours",
    "ClubParticipation", "MentorMeetings", "CounselingSessions",
    "IAT1", "IAT2", "IAT3"
]

@app.post("/predict")
def predict(data: AcademicInput):
    if model is None:
        raise HTTPException(status_code=503, detail="Model not loaded")

    try:
        # 1. Convert input to DataFrame
        input_dict = data.dict()
        df = pd.DataFrame([input_dict])

        # 2. Encode categorical features
        for col in CATEGORICAL_COLS:
            if col in df.columns:
                le = feature_encoders.get(col)
                if le:
                    df[col] = df[col].map(lambda x: transform_label(le, x))

        # 3. Ensure correct column order
        df = df[FEATURE_COLUMNS]

        # 4. Predict
        prediction_encoded = model.predict(df)[0]
        prediction_label = label_encoder.inverse_transform([prediction_encoded])[0]

        # 5. Get confidence
        confidence = 0.0
        if hasattr(model, 'predict_proba'):
            proba = model.predict_proba(df)[0]
            confidence = float(np.max(proba))

        # --- BUSINESS RULE OVERRIDE ---
        critical_triggers = []
```

```

    if data.Attendance < 75: critical_triggers.append("Attendance < 75%")
    if data.CGPA < 6.0: critical_triggers.append("CGPA < 6.0")
    if data.Backlogs >= 2: critical_triggers.append("Backlogs >= 2")

    if critical_triggers:
        prediction_label = "High"
        confidence = 0.99

    return {
        "prediction": str(prediction_label),
        "confidence": round(confidence, 2),
        "model": "xgboost_enhanced_v2"
    }
except Exception as e:
    raise HTTPException(status_code=500, detail=f"Prediction error: {str(e)}")

```

A.4.2 Explainable AI Service (xai-svc/service.py)

```

from fastapi import FastAPI
from pydantic import BaseModel
import pandas as pd
import joblib
import shap

app = FastAPI(title="XAI Service")

risk_model = joblib.load("../risk-svc/models/academic_risk_pipeline.pkl")

def explain_risk_model(input_df):
    prediction = risk_model.predict(input_df)[0]
    clf = risk_model.named_steps["clf"]

    # Feature importances
    if hasattr(clf, "feature_importances_"):
        importances = clf.feature_importances_
        feature_importance = {col: float(val) for col, val in zip(input_df.columns, importances)}
    else:
        feature_importance = {}

    warnings = []
    if input_df['Attendance'].iloc[0] < 70:
        warnings.append("[Warning] Low attendance.")
    if input_df['CGPA'].iloc[0] < 6.5:
        warnings.append("[Warning] CGPA below recommended level.")

    return {
        "prediction": int(prediction),

```

```

        "feature_importance": feature_importance,
        "warnings": warnings
    }

```

```

@app.post("/explain/risk")
def explain_risk(data: RiskInput):
    df = pd.DataFrame([data.dict()])
    return explain_risk_model(df)

```

A.4.3 Cognitive Service (cog-svc/service.py)

```

from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import pandas as pd

app = FastAPI()
models = {trait: joblib.load(f"models/{trait}_pipeline.pkl")
          for trait in ["openness", "conscientiousness", "extraversion",
                        "agreeableness", "neuroticism"]}

@app.post("/predict")
def predict_traits(input_data: SurveyInput):
    df = pd.DataFrame([input_data.dict()])
    # Scale to 0-100 and round to integer
    raw_predictions = {trait.capitalize(): float(models[trait].predict(df)[0]) for trait in models}
    predictions = {}
    for trait, value in raw_predictions.items():
        if value <= 1.0:
            predictions[trait] = int(value * 100)
        else:
            predictions[trait] = int(value)

    extended_profile = generate_extended_profile(predictions)
    return {"predictions": predictions, "extended_profile": extended_profile}

def generate_extended_profile(scores):
    openness = scores.get('Openness', 50)
    conscientiousness = scores.get('Conscientiousness', 50)
    extraversion = scores.get('Extraversion', 50)

    learning_style = {
        "visual": min(100, int(openness * 0.8 + 20)),
        "auditory": min(100, int(extraversion * 0.8 + 20)),
        "kinesthetic": min(100, int(conscientiousness * 0.8 + 20))
    }
    return {"learningStyle": learning_style}

```